Intro till Ansible

Arseniy Buzyan

Ansible Äger Skalan: Från En till Tusen

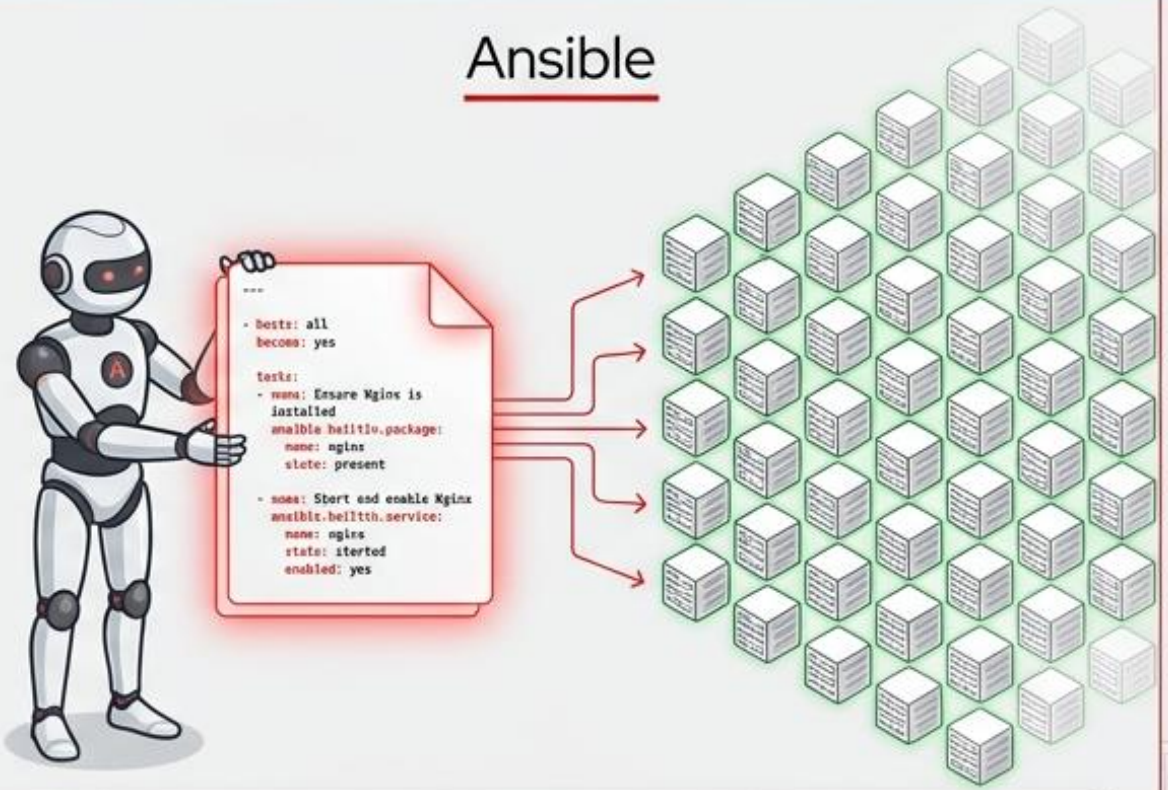
Manuell hantering



```
> _
```

Mänsklig interaktion. Begränsad av tid och risk för fel.

Ansible




```
> _
```

Ett öppen källkods-verktyg som fungerar som en robot. Du ger roboten ett recept (Playbooks skrivna i läsbart YAML-format), och den konfigurerar tusentals servrar exakt likadant, samtidiigt. Inga agenter krävs på målet.

Vad är Ansible? – Automatisering för Alla

ANSIBLE ÄR ETT ÖPPEN KÄLLKODS-VERKTYG FÖR ATT AUTOMATISERA KONFIGURATION, APPLIKATIONSDISTRIBUTION OCH MOLNPROVISIONERING.

ANSIBLE ÄR SOM EN "ROBOT" SOM KAN UTFÖRA SAMMA UPPGIFTER PÅ TUSENTALS SERVRAR SAMTIDIGT – UTAN ATT DU BEHÖVER INSTALLERA NÅGOT PÅ SERVRARNA I FÖRVÄG. ROBOTENS "RECEPT" KALLAS PLAYBOOKS OCH ÄR SKRIVNA I YAML-FORMAT, VILKET ÄR ETT LÄTTLÄST SPRÅK SOM LIKNAR VANLIG TEXT .

Från Manuell Smärta till Automatiserad Kontroll

Manuell IT-drift	Ansible Automation
Skalbarhet 100 servrar = 100 manuella insatser och inloggningar.	**Skalbarhet** Kör ett enda kommando, alla 100 servrar uppdateras parallellt.
Förutsägbarhet Extremt svårt att exakt återskapa en serverkonfiguration.	**Förutsägbarhet** Samma playbook ger exakt samma resultat varje gång.
Dokumentation Processer existerar ofta bara i en enskild teknikers huvud.	**Dokumentation** Infrastruktur som kod (IaC) – Playbooks fungerar som självdokumenterande kod.

De Tre Grundpelarna i Ansible

ansible-terminal — 3_pillars.yml



Agentlös

Ingen programvara eller agent behöver förinstalleras på målet. Ansible använder standardprotokoll (SSH för Linux, WinRM för Windows) för att ansluta och utföra uppgifter.

Enkelt

Infrastruktur beskrivs i YAML. Inga komplexa, proprietära skriptspråk att lära sig. Koden läses nästan som vanlig engelska.

Idempotent

Det mest kraftfulla konceptet. Du kan köra en playbook 100 gånger. Om servern redan har rätt konfiguration, görs inga ändringar – vilket förhindrar negativa bieffekter.

Bevisad ROI i Svensk Företagsmiljö

Skatteverket

(Värd för Sweden Ansible User Group).

Driver massiv automatisering och experimenterar aktivt med AI-drivna playbooks för framtidens IT-drift.

Volvo

Kombinerar Ansible och Terraform i en hybridstrategi för att automatisera komplex driftsättning.

ABB (Globalt & Svenskt)

Använder Red Hat Ansible Automation Platform för att transformera IT-leveransen. Ett enskilt användningsfall har sparat 900 000 SEK över tre år.

57h

Sparade timmar per månad, per team.

6,5h → 20m

Effektivisering av patching per server.

40 000h

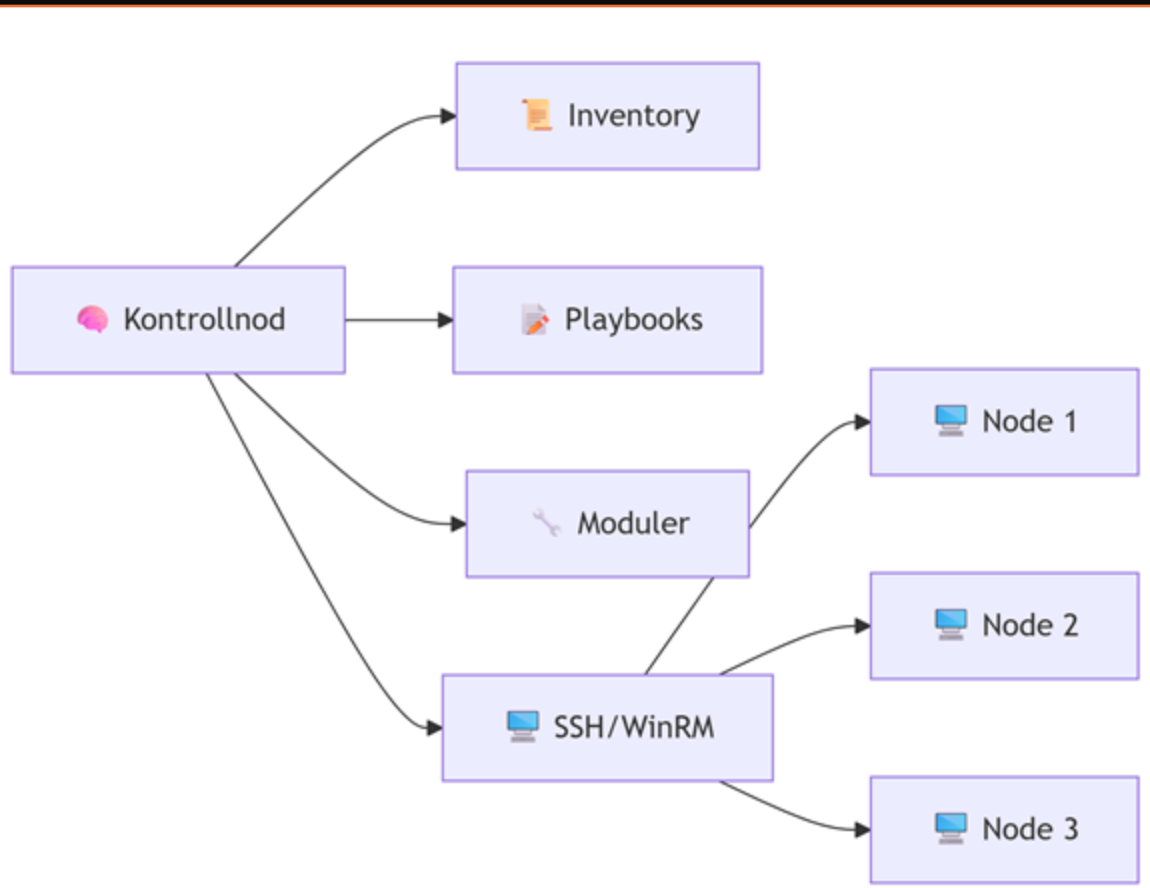
Frigjorda timmar från manuellt arbete.

4 000

Automatiserade ärenden via ServiceNow-integration.

Ansible Arkitektur – Tre Huvudkomponenter

Ansible har en enkel arkitektur med tre huvudkomponenter



De tre huvudkomponenterna

Enligt Ansible-dokumentationen: Kontrollnoden kan vara vilken dator som helst med Python installerat – men inte Windows utan WSL2 . Därför använder vi WSL2!

Komponent	Vad gör den?	Exempel
Kontrollnod	Datorn där Ansible är installerat – härifrån kör du kommandon	Din dator med WSL2
Inventory	En fil som listar vilka servrar som ska hanteras	web1.example.com
Moduler	Färdiga kodblock för specifika uppgifter	ping, apt, copy, service
Playbooks	YAML-filer med instruktioner – Ansible "recept"	install_webserver.yml

Ekosystemet:
Välj Rätt Verktyg för Rätt Uppgift

Verktyg	Huvudområde	Styrka
Ansible	OS-konfiguration & App-distribution	Agentlöst, idempotent, YAML.
Terraform	Provisionering (Skapa infrastruktur)	Skapar själva servrarna, nätverken och lagringen i molnet.
PowerShell	Windows-administration	Direkt, djupgående åtkomst till Windows-funktioner.

Branschrekommendation (Azure Local)

Har du mest Windows?
-> Börja med PowerShell + DSC.

Skapar du helt ny infrastruktur?
-> Använd Terraform.

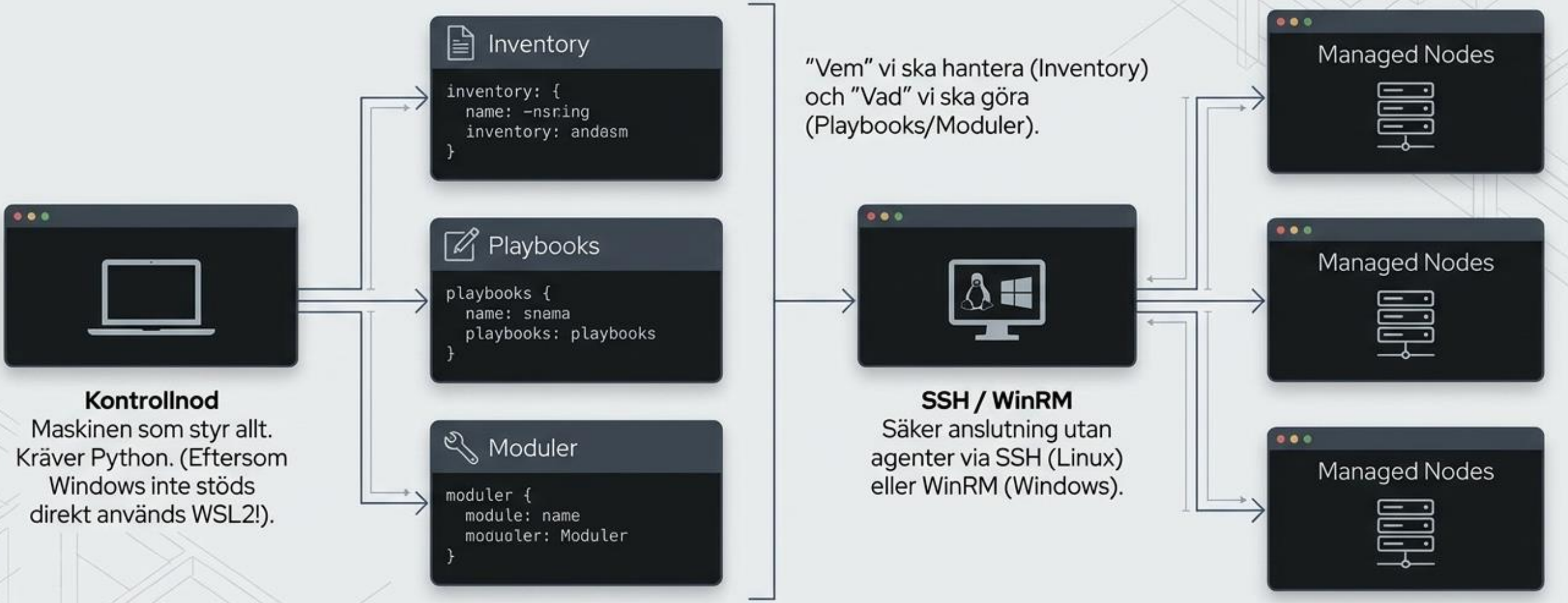
Blandade miljöer (Linux/Windows) som ska konfigureras?
-> Ansible är den gemensamma plattformen.

Rekommendation från branschen

Varför detta är viktigt: Använder du Ansible tillsammans med Terraform och PowerShell får du ett komplett automationsverktyg. Enligt en branschguide från Azure Local används Ansible främst för OS-konfiguration medan Terraform används för infrastruktur .

Situation	Rekommendation
Du har mest Windows-servrar	Börja med PowerShell + DSC
Du har blandade miljöer (Windows + Linux)	Använd Ansible som gemensam plattform
Du skapar servrar i molnet	Använd Terraform för provisionering

Arkitekturen Bakom Automationen



Kontrollnod
Maskinen som styr allt. Kräver Python. (Eftersom Windows inte stöds direkt används WSL2!).

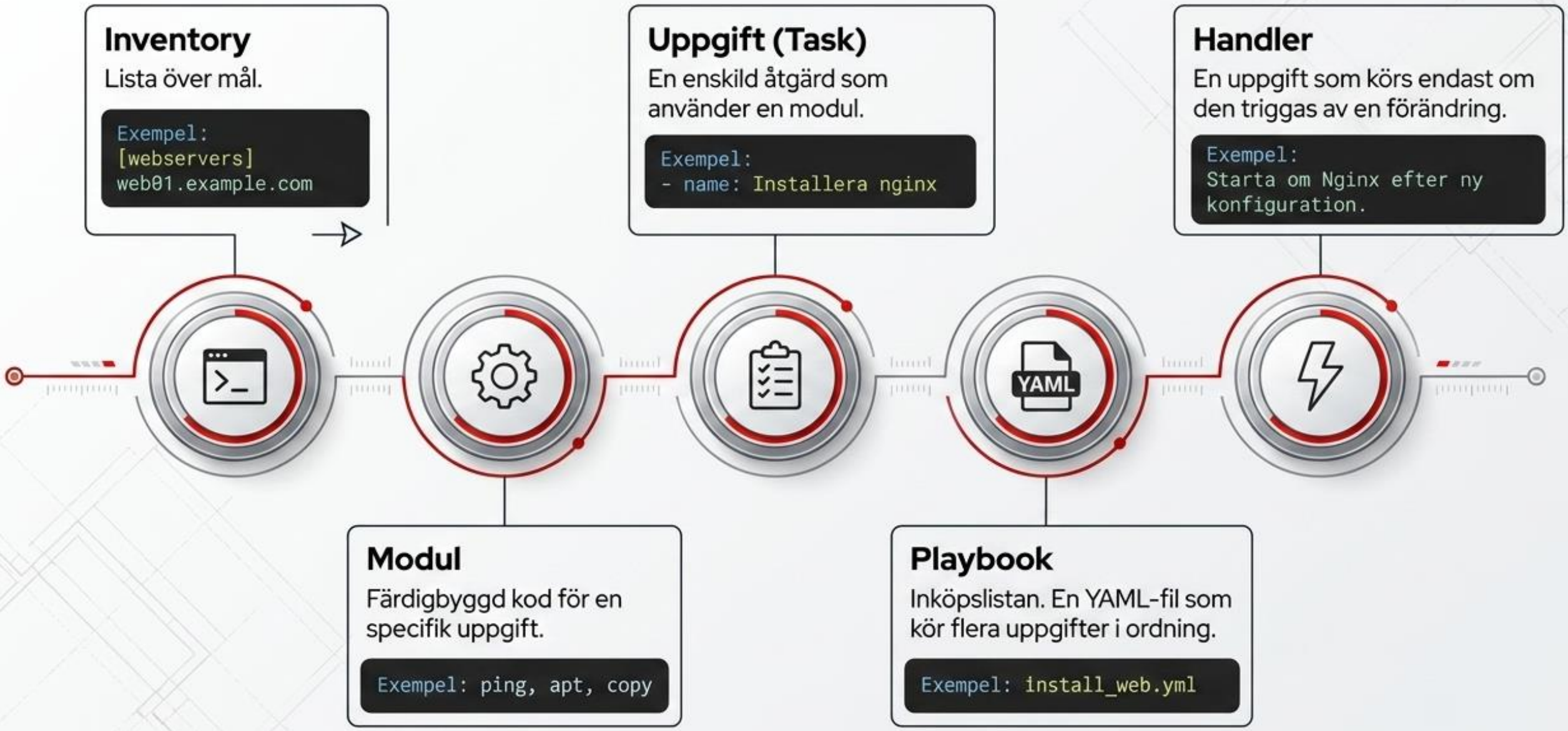
SSH / WinRM
Säker anslutning utan agenter via SSH (Linux) eller WinRM (Windows).

Målen som konfigureras (webbservrar, databaser, etc).

Ansible-koncept – Ordlista för Nybörjare

Tänk: En playbook är som en inköpslista med instruktioner – och Ansible utför alla uppgifter i listan i ordning.

En Visuell Ordlista för Ansible



Här är de viktigaste Ansible-begreppen förklaras enkelt

Begrepp	Enkel förklaring	Exempel
Kontrollnod	Din dator där Ansible är installerat	Din WSL2-miljö
Managed node (värd)	En server som Ansible hanterar	web01.example.com
Inventory	Lista över servrar som ska hanteras	[webservers] web01.example.com
Modul	Färdig kod för en specifik uppgift	ping, apt, service, copy
Uppgift	En enskild åtgärd i en playbook	- name: Installera nginx
Playbook	En YAML-fil med flera uppgifter	install_webserver.yml
Handler	Körs bara om den blir notifierad	Starta om nginx efter konfigurationsändring

Installera Ansible i WSL2 – Steg-för-Steg

Ansible fungerar inte direkt i Windows – WSL2 är den rekommenderade plattformen.

Steg 1 – Installera WSL2 (PowerShell som Admin)

- `bash`
- `wsl --install`
- `wsl --install -d Ubuntu-24.04`

Installera Ansible (inuti WSL2)

```bash
sudo apt update && sudo apt upgrade -y
sudo apt install -y ansible
```

Ansible fungerar inte direkt i Windows. Din lokala kontrollnod kräver Python (v3.12-3.14). Lösningen är Windows Subsystem for Linux (WSL2).

PowerShell - Installera WSL2

```
> wsl --install
> wsl --install -d Ubuntu-24.04

(Körs i PowerShell som Administratör)
```

Ubuntu - Installera Ansible

```
$ sudo apt update && sudo apt upgrade -y
$ sudo apt install -y ansible
$ ansible --version

(Verifiera installationen)
```

Steg 3 – Verifiera installationen

```bash
ansible --version
```

Steg 4 – Testa att Ansible fungerar

```bash
ansible localhost -m ping
```

Förväntat resultat:

Krav: Python 3.12-3.14 på kontrollnoden och Python 3.9-3.14 på managed nodes.

```
text
localhost | SUCCESS => {
    "changed": false,
    "ping": "pong"
}
```

Din Första Inventory

Inventory = en fil som talar om för Ansible vilka servrar som ska hanteras.

- **Skapa** `inventory.ini`**:**
- ini

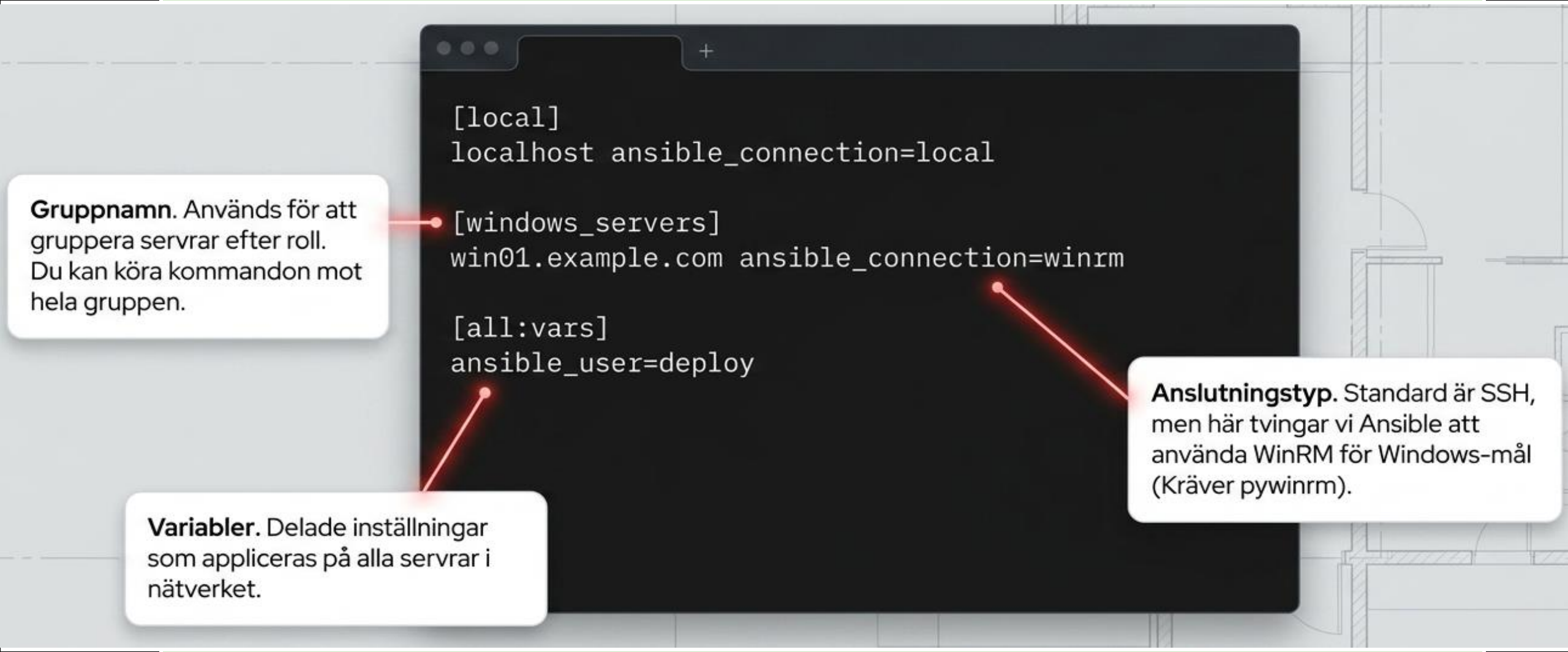
```ini
# inventory.ini - Enkel inventory-fil

[webservers]
web01.example.com
web02.example.com
192.168.1.10

[databases]
db01.example.com

[windows_servers]
win01.example.com ansible_connection=winrm

[all:vars]
ansible_user=deploy
```


[local]
localhost ansible_connection=local
[windows_servers]
win01.example.com ansible_connection=winrm
[all:vars]
ansible_user=deploy
Gruppnamn. Används för att gruppera servrar efter roll. Du kan köra kommandon mot hela gruppen.
Anslutningstyp. Standard är SSH, men här tvingar vi Ansible att använda WinRM för Windows-mål (Kräver pywinrm).
Variabler. Delade inställningar som appliceras på alla servrar i nätverket.

Viktiga delar

Del	Förklaring	Exempel
[gruppnamn]	Gruppera servrar efter roll	[webservers]
ansible_connection	Anger hur Ansible ska ansluta	local, ssh, winrm
[all:vars]	Gemensamma variabler för alla servrar	ansible_user=deploy

Testa inventory

Enligt Linux Server Admin Wiki: För Windows-värdar behöver du lägga till ansible_connection=winrm och installera pywinrm.

bash

```
ansible all -i inventory.ini -m ping
```

Ad-hoc-kommandon – Kör Enkla Kommandon

- Ad-hoc-kommandon = Kör ett enskilt kommando på servrar utan playbook.
- Grundläggande syntax:
- bash
- `ansible <värdgrupp> -m <modul> -a <argument>`

Exempel

Kommando	Vad gör det?
ansible all -m ping	Testar anslutning till alla servrar
ansible webservers -m shell -a "uptime"	Kör uptime på alla webbservrar
ansible webservers -m apt -a "name=nginx state=present"	Installerar nginx på alla webbservrar (Debian/Ubuntu)
ansible windows -m win_ping	Testar anslutning till Windows-servrar

När använder man ad-hoc-kommandon?

- Vid snabba kontroller (t.ex. pinga servrar)
- Vid engångsåtgärder (t.ex. installera ett paket snabbt)
- Enligt en kursbeskrivning: Ad-hoc-kommandon är bra för enkla uppgifter, men playbooks är bättre när du vill återanvända och dokumentera dina automationer.

Arbetsmetoder: Ad-Hoc vs. Playbooks



Ad-Hoc Kommandon

- **När?** Snabba kontroller, felsökning, eller engångsåtgärder.
- **Syntax:** ansible <värdgrupp> -m <modul> -a <argument>

```
ansible all -m ping
ansible webservers -m shell -a "uptime"
```

Playbooks

- **När?** Återanvändbar, dokumenterad och komplex automation. Infrastruktur som kod.
- **Syntax:** YAML-filer.

```
ansible-playbook -i inventory.ini site.yml
```

Din Första Playbook

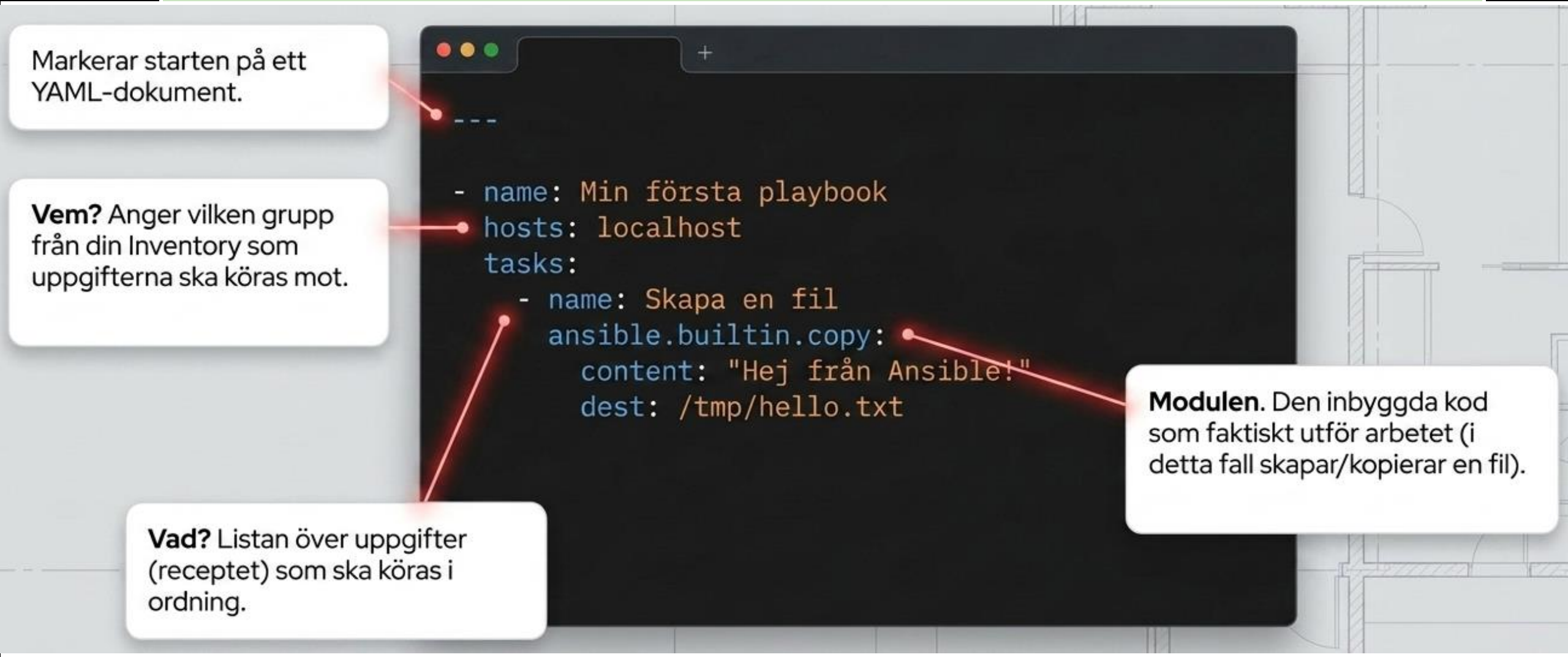
En playbook är en YAML-fil med instruktioner – Ansible "recept".

Skapa `first_playbook.yml`**:**

- yaml

```yaml
---
- name: Min första playbook
  hosts: localhost
  tasks:
    - name: Skapa en fil
      ansible.builtin.copy:
        content: "Hej från Ansible!"
        dest: /tmp/hello.txt

    - name: Visa meddelande
      ansible.builtin.debug:
        msg: "Filen skapades!"
```


Markerar starten på ett YAML-dokument.
Vem? Anger vilken grupp från din Inventory som uppgifterna ska köras mot.
Vad? Listan över uppgifter (receptet) som ska köras i ordning.
Modulen. Den inbyggda kod som faktiskt utför arbetet (i detta fall skapar/kopierar en fil).

- name: Min första playbook
hosts: localhost
tasks:
- name: Skapa en fil
ansible.builtin.copy:
content: "Hej från Ansible!"
dest: /tmp/hello.txt

Kör playbook

```
bash

ansible-playbook -i inventory.ini
first_playbook.yml
```



Grön
(ok=3)

SUCCESS / OK. Målet var nåbart och uppgiften krävde ingen ändring eftersom rätt tillstånd redan var uppfyllt (Idempotent!).



Gul
(changed=1)

CHANGED. Ansible anslöt och var tvungen att göra en förändring på servern för att uppfylla din playbook.



Röd
(failed=0)

FAILED. Något gick fel (t.ex. nätverksfel eller felaktiga behörigheter), och uppgiften avbröts.

Vad händer i output?

Tänk: Playbooks är idempotenta – kör du samma playbook två gånger gör den inga ändringar andra gången.

Vad händer i output?

- `ok=3` betyder att tre uppgifter kördes utan ändringar
- `changed=0` betyder att inga ändringar gjordes
- `failed=0` betyder att inga fel inträffade

Workshop – Din Första Ansible-Playbook

Mål: Skapa och köra din första Ansible-playbook.

Steg 1 – Skapa inventory

ini

```ini
# inventory.ini

[local]

localhost ansible_connection=local
```

Steg 2 – Skapa hello.yml

```yaml
---
- name: Hello Ansible
  hosts: localhost
  tasks:
    - name: Skapa en fil
      ansible.builtin.copy:
        content: "Hej från Ansible!"
        dest: /tmp/ansible_hello.txt

    - name: Visa meddelande
      ansible.builtin.debug:
        msg: "Filen skapades i
```

Steg 3 – Kör playbook

```bash
ansible-playbook -i inventory.ini hello.yml
```

Steg 4 – Verifiera resultat

```bash
cat /tmp/ansible_hello.txt
```

Diskussionsfråga: Vad händer om du kör ansible-playbook -i inventory.ini hello.yml en gång till?

Det Stora Testet och Sammanfattning



Svaret: Ingenting. Output blir helt grön (**changed=0**). Eftersom filen **/tmp/hello.txt** redan existerar, förstår Ansible (**Idempotens**) att systemet redan är i rätt tillstånd. Automationen är 100% säker.

- **Agentlöst & Säkert:** Styr all infrastruktur via standardprotokoll utan extra programvara.
- **Bevisad ROI:** Frigör tusentals timmar och minska konfigurationstider från timmar till minuter (bevisat av ABB, Volvo, m.fl.).
- **Redo att Skala:** Med WSL2 uppsatt och YAML-grunderna på plats är du redo att bygga infrastruktur som kod.

Sammanfattning

- **Vad Ansible är**: Agentlöst automationsverktyg för konfiguration, applikationsdistribution och provisionering
- **Varför Ansible**: Enkelt (YAML), idempotent, agentlöst – och används i svenska företag som ABB, Volvo och Skatteverket
- **Arkitektur**: Kontrollnod, Inventory, Moduler, Playbooks
- **Installation i WSL2**: sudo apt install ansible – Ansible fungerar inte direkt i Windows utan WSL2
- **Ad-hoc-kommandon**: Enstaka kommandon på servrar – ansible webservers -m ping
- **Playbooks**: YAML-filer med instruktioner – idempotenta och återanvändbara

Quiz

https://forms.gle/3hvJWBVeU3Bs265h7